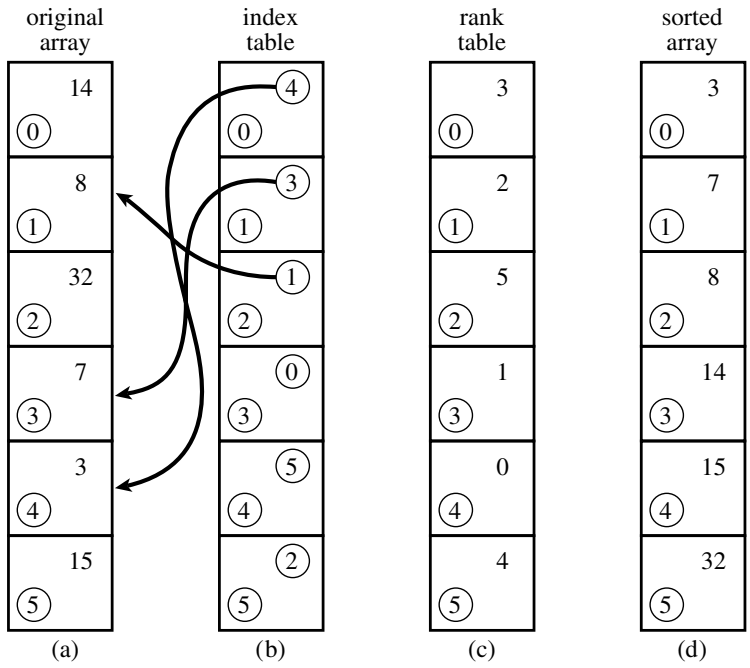

Figure 8.4.1. (a) An unsorted array of six numbers. (b) Index table whose entries are pointers to the elements of (a) in ascending order. (c) Rank table whose entries are the ranks of the corresponding elements of (a). (d) Sorted array of the elements in (a).

to the left of the kth, all larger elements to the right, and scrambling the order within each subset. This side effect is at best innocuous, at worst downright inconvenient. When an array is very long, so that making a scratch copy of it is taxing on memory, one turns to *in-place* algorithms without side effects, which are slower but leave the original array undisturbed.

The most common use of selection is in the statistical characterization of a set of data. One often wants to know the median element in an array (quantile $p = 1/2$) or the top and bottom quartile elements (quantile $p = 1/4, 3/4$). When N is odd, the exact definition of the median is that it is the kth element, with $k = (N-1)/2$. When N is even, statistics books define the median as the arithmetic mean of the elements $k = N/2 - 1$ and $k = N/2$ (that is, $N/2$ from the bottom and $N/2$ from the top). If you embrace such formality, you must perform two separate selections to find these elements. (If you do the first selection by a partition method, see below, you can do the second by a single pass through $N/2$ elements in the right partition, looking for the smallest element.) For $N > 100$ we usually use $k = N/2$ as the median element, formalists be damned.

A variant on selection for large data sets is *single-pass selection*, where we have a stream of input values, each of which we get to see only once. We want to be able to report at any time, say after N values, the kth smallest (or largest) value seen so far, or, equivalently, the quantile value for some p. We will describe two approaches: If we care only about the smallest (or largest) M values, for fixed M, so that $0 \le k < M$, then there are good algorithms that require only M storage. On the other hand, if we can tolerate an approximate answer, then there are efficient

algorithms that can report at any time a good *estimate* of the p-quantile value for any p, $0 < p < 1$. That is to say, we will get not the exact kth smallest element, $k = pN$, of the N that have gone by, but something very close to it — and without requiring N storage or having to know p in advance.

The fastest general method for selection, allowing rearrangement, is *partitioning*, exactly as was done in the Quicksort algorithm (§8.2). Selecting a "random" partition element, one marches through the array, forcing smaller elements to the left, larger elements to the right. As in Quicksort, it is important to optimize the inner loop, using "sentinels" (§8.2) to minimize the number of comparisons. For sorting, one would then proceed to further partition both subsets. For selection, we can ignore one subset and attend only to the one that contains our desired kth element. Selection by partitioning thus does not need a stack of pending operations, and its operations count scales as N rather than as $N \log N$ (see [1]). Comparison with `sort` in §8.2 should make the following routine obvious.

sort.h

```
template<class T>
T select(const Int k, NRvector<T> &arr)
```

Given `k` in `[0..n-1]` returns an array value from `arr[0..n-1]` such that `k` array values are less than or equal to the one returned. The input array will be rearranged to have this value in location `arr[k]`, with all smaller elements moved to `arr[0..k-1]` (in arbitrary order) and all larger elements in `arr[k+1..n-1]` (also in arbitrary order).

```
{
    Int i,ir,j,l,mid,n=arr.size();
    T a;
    l=0;
    ir=n-1;
    for (;;) {
        if (ir <= l+1) {                    Active partition contains 1 or 2 elements.
            if (ir == l+1 && arr[ir] < arr[l])        Case of 2 elements.
                SWAP(arr[l],arr[ir]);
            return arr[k];
        } else {
            mid=(l+ir) >> 1;                Choose median of left, center, and right el-
            SWAP(arr[mid],arr[l+1]);            ements as partitioning element a. Also
            if (arr[l] > arr[ir])               rearrange so that arr[l] ≤ arr[l+1],
                SWAP(arr[l],arr[ir]);           arr[ir] ≥ arr[l+1].
            if (arr[l+1] > arr[ir])
                SWAP(arr[l+1],arr[ir]);
            if (arr[l] > arr[l+1])
                SWAP(arr[l],arr[l+1]);
            i=l+1;                          Initialize pointers for partitioning.
            j=ir;
            a=arr[l+1];                     Partitioning element.
            for (;;) {                      Beginning of innermost loop.
                do i++; while (arr[i] < a);         Scan up to find element > a.
                do j--; while (arr[j] > a);         Scan down to find element < a.
                if (j < i) break;           Pointers crossed. Partitioning complete.
                SWAP(arr[i],arr[j]);
            }                               End of innermost loop.
            arr[l+1]=arr[j];                Insert partitioning element.
            arr[j]=a;
            if (j >= k) ir=j-1;             Keep active the partition that contains the
            if (j <= k) l=i;                    kth element.
        }
    }
}
```

If you don't want your array `arr` to be rearranged, then you will want to make

a scratch copy before calling `select`, e.g.,

```
VecDoub brr(arr);
```

The reason for not doing this internally in `select` is because you may wish to call `select` with a variety of different values `k`, and it would be wasteful to copy the vector anew each time; instead, just let `brr` keep getting rearranged.

8.5.1 Tracking the M Largest in a Single Pass

Of course `select` should not be used for the trivial cases of finding the largest, or smallest, element in an array. Those cases, you code by hand as simple `for` loops.

There are also efficient ways to code the case where k is bounded by some fixed M, modest in comparison to N, so that memory of order M is not burdensome. Indeed, N may not even be known: You may have a stream of incoming data values and be called upon at any time to provide a list of the M largest values seen so far.

A good approach to this case is to use the method of Heapsort (§8.3), maintaining a heap of the M largest values. The advantage of the heap structure, as opposed to a linear array of length M, is that at most $\log M$, rather than M, operations are required every time a new data value is processed.

The object `Heapselect` has a constructor, by which you specify M, an "`add`" method that assimilates a new data value, and a "`report`" method for getting the kth largest seen so far. Note that the initial cost of a report is $O(M \log M)$, because we need to sort the heap; but you can then get all values of k at no extra cost, until you do the next `add`. A special case is that getting the $M-1$st largest is always cheap, since it is at the top of the heap; so if you have a single favorite value of k, it is best to choose M with $M - 1 = k$.

sort.h

```
struct Heapselect {
```

Object for tracking the `m` largest values seen thus far in a stream of values.

```
    Int m,n,srtd;
    VecDoub heap;

    Heapselect(Int mm) : m(mm), n(0), srtd(0), heap(mm,1.e99) {}
```

Constructor. The argument is the number of largest values to track.

```
    void add(Doub val) {
```

Assimilate a new value from the stream.

```
        Int j,k;
        if (n<m) {                          Heap not yet filled.
            heap[n++] = val;
            if (n==m) sort(heap);           Create initial heap by overkill!
        } else {
            if (val > heap[0]) {            Put it on the heap?
                heap[0]=val;
                for (j=0;;) {               Sift down.
                    k=(j << 1) + 1;
                    if (k > m-1) break;
                    if (k != (m-1) && heap[k] > heap[k+1]) k++;
                    if (heap[j] <= heap[k]) break;
                    SWAP(heap[k],heap[j]);
                    j=k;
                }
            }
            n++;
        }
        srtd = 0;                           Mark heap as "unsorted".
    }
```

```
    Doub report(Int k) {
    Return the kth largest value seen so far. k=0 returns the largest value seen, k=1 the second
    largest, ... , k=m-1 the last position tracked. Also, k must be less than the number of
    previous values assimilated.
        Int mm = MIN(n,m);
        if (k > mm-1) throw("Heapselect k too big");
        if (k == m-1) return heap[0];          Always free, since top of heap.
        if (! srtd) { sort(heap); srtd = 1; } Otherwise, need to sort the heap.
        return heap[mm-1-k];
    }
};
```

8.5.2 Single-Pass Estimation of Arbitrary Quantiles

The data values fly by in a stream. You get to look at each value only once, and do a constant-time process on it (meaning that you can't take longer and longer to process later and later data values). Also, you have only a fixed amount of storage memory. From time to time you want to know the median value (or 95th percentile value, or arbitrary p-quantile value) of the data that you have seen thus far. How do you do this?

Evidently, with the conditions stated, you'll have to tolerate an approximate answer, since an exact answer must require unbounded storage and (perhaps) unlimited processing. If you think that "binning" is somehow the answer, you are right. But it is not immediately obvious how to choose the bins, since you have to see a potentially unlimited amount of data before you can tell for sure how its values are distributed.

Chambers et al. [2] have given a robust, and extremely fast, algorithm, which they call *IQ agent*, that adaptively adjusts a set of bins so that they converge to the data values of specified quantile p-values. The general idea (see Figure 8.5.1) is to accumulate incoming data into batches, then to update a stored, piecewise linear, cumulative distribution function (cdf) by adding a batch's cdf and then interpolating back to a fixed set of p-values. Arbitrary requested quantile values ("incremental quantiles," or "IQs," hence the algorithm's name) can be obtained at any time by linear interpolation on the stored cdf. Batching allows the program to be very efficient, with an (amortized) cost of only a small number of operations per new data value. The batching is done transparently to the user.

Similar to `Heapselect`, the `IQagent` object has `add` and `report` methods, the latter now taking a value for p as its argument. In the implementation below, we use a batch size of `nbuf=1000` but do an early update step with a partial batch whenever a quantile is requested. With these parameters, you should therefore request quantile information no more frequently than after every few `nbuf` data values, at which point you can request as many different values of p as you want before continuing. The alternative is to remove the call to `update` from `report`, in which case you'll get fast, but constant, answers, changing only after each regular batch update.

`IQagent` uses internally a general purpose set of 251 p-values that includes integer percentile points from 10 to 90, and a logarithmically spaced set of smaller and larger values spanning 10^{-6} to $1 - 10^{-6}$. Other p-values that you request are obtained by interpolation. Of course you cannot get meaningful tail quantiles for small values of p until at least several times $1/p$ data values have been processed. Before that, the program will simply report the smallest value previously seen (or largest value previously seen, for $p \to 1$).

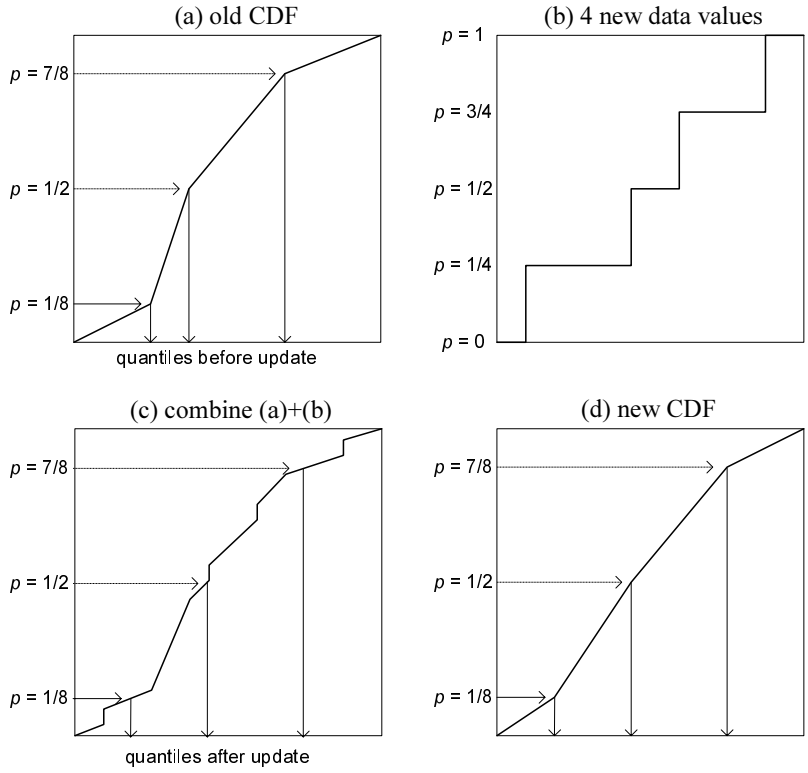

Figure 8.5.1. Algorithm for updating a piecewise linear cumulative distribution function (cdf). (a) The cdf is represented by quantile values at a fixed set of p-values (here, just 3). (b) A batch of new data values (here, just 4) define a stepwise constant cdf. (c) The two cdfs are summed. New data steps are small in proportion to the new batch size versus number of data values previously processed. (d) The new cdf representation is obtained by interpolating the fixed p-values to (c).

iqagent.h

```
struct IQagent {
```

Object for estimating arbitrary quantile values from a continuing stream of data values.

```
    static const Int nbuf = 1000;
```

Batch size. You may $\times 10$ if you expect $> 10^6$ data values.

```
    Int nq, nt, nd;
    VecDoub pval,dbuf,qile;
    Doub q0, qm;

    IQagent() : nq(251), nt(0), nd(0), pval(nq), dbuf(nbuf),
    qile(nq,0.), q0(1.e99), qm(-1.e99) {
```

Constructor. No arguments.

```
        for (Int j=85;j<=165;j++) pval[j] = (j-75.)/100.;
```

Set general purpose array of p-values ranging from 10^{-6} to $1-10^{-6}$. You can change this if you want:

```
        for (Int j=84;j>=0;j--) {
            pval[j] = 0.87191909*pval[j+1];
            pval[250-j] = 1.-pval[j];
        }
    }

    void add(Doub datum) {
```

Assimilate a new value from the stream.

```
        dbuf[nd++] = datum;
        if (datum < q0) {q0 = datum;}
        if (datum > qm) {qm = datum;}
```

```
        if (nd == nbuf) update();                Time for a batch update.
    }

    void update() {
```
Batch update, as shown in Figure 8.5.1. This function is called by add or report and should not be called directly by the user.
```
        Int jd=0,jq=1,iq;
        Doub target, told=0., tnew=0., qold, qnew;
        VecDoub newqile(nq);                     Will be new quantiles after update.
        sort(dbuf,nd);
        qold = qnew = qile[0] = newqile[0] = q0;     Set lowest and highest to min
        qile[nq-1] = newqile[nq-1] = qm;                 and max values seen so far,
        pval[0] = min(0.5/(nt+nd),0.5*pval[1]);          and set compatible p-values.
        pval[nq-1] = max(1.-0.5/(nt+nd),0.5*(1.+pval[nq-2]));
        for (iq=1;iq<nq-1;iq++) {                Main loop over target p-values for inter-
            target = (nt+nd)*pval[iq];               polation.
            if (tnew < target) for (;;) {
```
Here's the guts: We locate a succession of abscissa-ordinate pairs (qnew,tnew) that are the discontinuities of value or slope in Figure 8.5.1(c), breaking to perform an interpolation as we cross each target.
```
                if (jq < nq && (jd >= nd || qile[jq] < dbuf[jd])) {
```
Found slope discontinuity from old CDF.
```
                    qnew = qile[jq];
                    tnew = jd + nt*pval[jq++];
                    if (tnew >= target) break;
                } else {                         Found value discontinuity from batch data
                    qnew = dbuf[jd];                 CDF.
                    tnew = told;
                    if (qile[jq]>qile[jq-1]) tnew += nt*(pval[jq]-pval[jq-1])
                        *(qnew-qold)/(qile[jq]-qile[jq-1]);
                    jd++;
                    if (tnew >= target) break;
                    told = tnew++;
                    qold = qnew;
                    if (tnew >= target) break;
                }
                told = tnew;
                qold = qnew;
            }                                    Break to here and perform the new interpolation.
            if (tnew == told) newqile[iq] = 0.5*(qold+qnew);
            else newqile[iq] = qold + (qnew-qold)*(target-told)/(tnew-told);
            told = tnew;
            qold = qnew;
        }
        qile = newqile;
        nt += nd;
        nd = 0;
    }

    Doub report(Doub p) {
```
Return estimated p-quantile for the data seen so far. (E.g., p = 0.5 for median.)
```
        Doub q;
        if (nd > 0) update();                    You may want to remove this line. See text.
        Int jl=0,jh=nq-1,j;
        while (jh-jl>1) {                        Locate place in table by bisection.
            j = (jh+jl)>>1;
            if (p > pval[j]) jl=j;
            else jh=j;
        }
        j = jl;                                  Interpolate.
        q =  qile[j] + (qile[j+1]-qile[j])*(p-pval[j])/(pval[j+1]-pval[j]);
        return MAX(qile[0],MIN(qile[nq-1],q));
    }
};
```

How accurate is the IQ agent algorithm, as compared, say, to storing all N data values in an array A and then reporting the "exact" quantile $A_{\lfloor pN \rfloor}$? There are several sources of error, all of which you can control by modifying parameters in `IQagent`. (We think that the default parameters will work just fine for almost all users.) First, there is interpolation error: The desired cdf is represented by a piecewise linear function between `nq=251` stored values. For typical distributions, this limits the accuracy to three or four significant figures. We find it hard to believe that anyone needs to know a median, e.g., more accurately than this, but if you do, then you can increase the density of p-values in the regions of interest.

Second, there are statistical errors. One way to characterize these is to ask what value j has A_j closest to the quantile reported by IQ agent, and then how small is $|j - pN|$ as a fraction of $[Np(1-p)]^{1/2}$, the accuracy inherent in your finite sample size N. If this fraction is $\lesssim 1$, then the estimate is "good enough," meaning that no method can do substantially better at estimating the population quantiles given your sample.

With the default parameters, and for reasonably behaved distributions, `IQagent` passes this test for $N \lesssim 10^6$. For larger N, the statistical error becomes significant (though still generally smaller than the interpolation error, above). You can, however, decrease it by increasing the batch size, `nbuf`. Larger is always better, if you have the memory and can tolerate the logarithmic increase in the cost per point of the sort.

Although the accuracy of `IQagent` is not guaranteed by a provable bound, the algorithm is fast, robust, and highly recommended. For other approaches to incremental quantile estimation, including some that do give provable bounds (but have other issues), see [3,4] and references cited therein.

8.5.3 Other Uses for Incremental Quantile Estimation

Incremental quantile estimation provides a useful way to histogram data into variable-size bins that each contain the same number of points, without knowing in advance the bin boundaries: First, throw N data values at an `IQagent` object. Next, choose a number of bins m, and define

$$p_i \equiv \frac{i}{m}, \qquad i = 0, \ldots, m \tag{8.5.1}$$

Finally, if q_i is the quantile value at p_i, plot the ith bin from q_i to q_{i+1} with a height

$$h_i = N \frac{p_{i+1} - p_i}{q_{i+1} - q_i}, \qquad i = 0, \ldots, m-1 \tag{8.5.2}$$

A different application concerns the monitoring of quantile values for changes. For example, you might be producing widgets with a parameter T whose tolerance is $T \pm \delta T$, and you want an early warning if the observed values of T at the 5th and 95th percentiles start to drift.

The `IQagent` object is easily modified for such applications. Simply change the line `nt += nd` to `nt = my_constant`, where `my_constant` is the number of past widgets that you want to average over. (More precisely, the number corresponding to one e-fold of weight decrease in an exponentially decreasing average over all past production.) Now, the stored cdf combines with a new batch of data with a constant, not an increasing, weight, and you can look for changes over time in any desired quantiles.